IT302 Project Report on

Predictive Serverless Cold-Start Optimizer

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY

By

Rahul Saravanan (231IT054)

Shishir Hegde (231IT067)



Department of Information Technology
National Institute of Technology Karnataka, Surathkal
Mangalore – 575025

November, 2025

Abstract

Serverless applications often spend a substantial part of their first-request latency loading dependencies. This paper presents the Predictive Cold-Start Optimizer (PCSO), an application-level approach that combines static dependency analysis with runtime transition prediction. The static phase separates startup-critical imports from request-specific imports. The runtime phase learns likely endpoint transitions and preloads the predicted dependency closure before the next request arrives. Against an eager no-optimizer baseline, the evaluated system reduces blended code-loading latency from 31.36 ms to 4.77 ms and total response latency from 35.29 ms to 15.48 ms. These correspond to reductions of 84.8% and 56.1%, respectively. The study also compares the static-only FaaSLight-style stage, which provides a useful minimum-improvement reference but can shift heavy import cost into handler execution.

Index Terms– serverless computing, cold start, dependency analysis, lazy loading, transition prediction, preloading.

I. Introduction

Function-as-a-Service platforms create execution environments on demand. A newly created environment must initialize application code before it can serve its first request. The initialization cost is especially visible in services that include database clients, SDKs, model wrappers, or media-processing libraries. Loading every dependency eagerly is simple, but it makes optional functionality part of the critical path.

This work evaluates a progression from an eager baseline to a final Predictive Cold-Start Optimizer (PCSO). The comparison has three points: the true no-optimizer baseline, a static-only V1 stage inspired by FaaSLight [1], and PCSO. The purpose of retaining V1 is not to present it as the final system, but to show the minimum improvement obtained from static deferral before prediction and preloading are added.

The contributions are:

- an AST-based dependency classification method for identifying startup-critical and deferrable imports;
- an online first-order transition model for predicting the next endpoint;
- a preloading policy that warms the predicted dependency closure before the next request; and
- a three-way evaluation across light, medium, heavy, and blended workload profiles.

II. System Design

A. Static dependency analysis

The analyzer inspects the source structure of each function. Imports at module scope are classified as indispensable because they are needed to construct the handler environment. Imports inside functions or conditional branches are classified as optional candidates for deferral. This classification is conservative: it changes when a dependency is loaded, not what the handler computes.

B. Runtime transition prediction

PCSO maintains a first-order Markov transition table. For an observed transition from endpoint A to endpoint B, it increments $N(A, B)$. The probability of the next endpoint is estimated as

$$P(B \mid A) = \frac{N(A, B)}{\sum_x N(A, x)}. \tag{1}$$

TABLE I
Compared runtime versions.

Version	Policy
Baseline	Eagerly loads indispensable and optional dependencies.
V1 / FaaSLight	Loads indispensable imports; optional imports remain handler-lazy.
PCSO	V1 policy plus transition prediction and dependency-closure preloading.

The highest-probability endpoint is selected only when its confidence reaches the configured threshold. PCSO then warms the predicted endpoint and the local dependencies identified by the static analysis. If the confidence is low or the request differs from the prediction, execution falls back to the static-loading behavior; prediction does not change functional correctness.

III. Evaluation Methodology

The evaluation uses four workload profiles. Tier 1 represents lightweight CRUD and utility functions. Tier 2 represents medium enterprise services with authentication, validation, and persistence logic. Tier 3 represents heavy OCR, fraud-analysis, and media-processing pipelines. The blended profile samples 50% Tier 1, 35% Tier 2, and 15% Tier 3 activity.

Each tiered profile contains 24 invocations and the blended profile contains 32. Fixed random seeds make the stochastic sequences repeatable. Each version is evaluated in an isolated runtime state. The reported metrics are mean import latency, mean handler latency, mean total response latency, p95 latency, resident memory, and predictive cache-hit rate. Background preloading is measured separately and is not added to the response time of the request that triggered it.

IV. Results

A. Tier 3: why PCSO gains over V1

The Tier 3 result exposes the difference between deferral and preloading. V1 reduces code-loading latency from 81.15 ms to 17.05 ms, but the heavy OCR, machine-learning, and media dependencies are then imported inside the handler. Consequently, total latency changes only from 89.47 ms to 88.98 ms. PCSO uses the observed transition history to warm the predicted heavy dependency closure before the request. Total Tier 3 latency therefore falls to

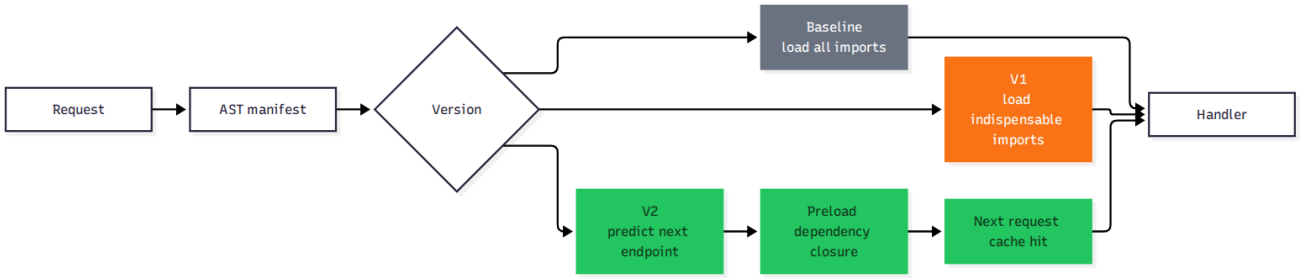


Fig. 1. PCSO combines a static dependency manifest with runtime prediction. The eager baseline loads the full dependency set, V1 loads only indispensable imports, and PCSO additionally warms the predicted next endpoint.

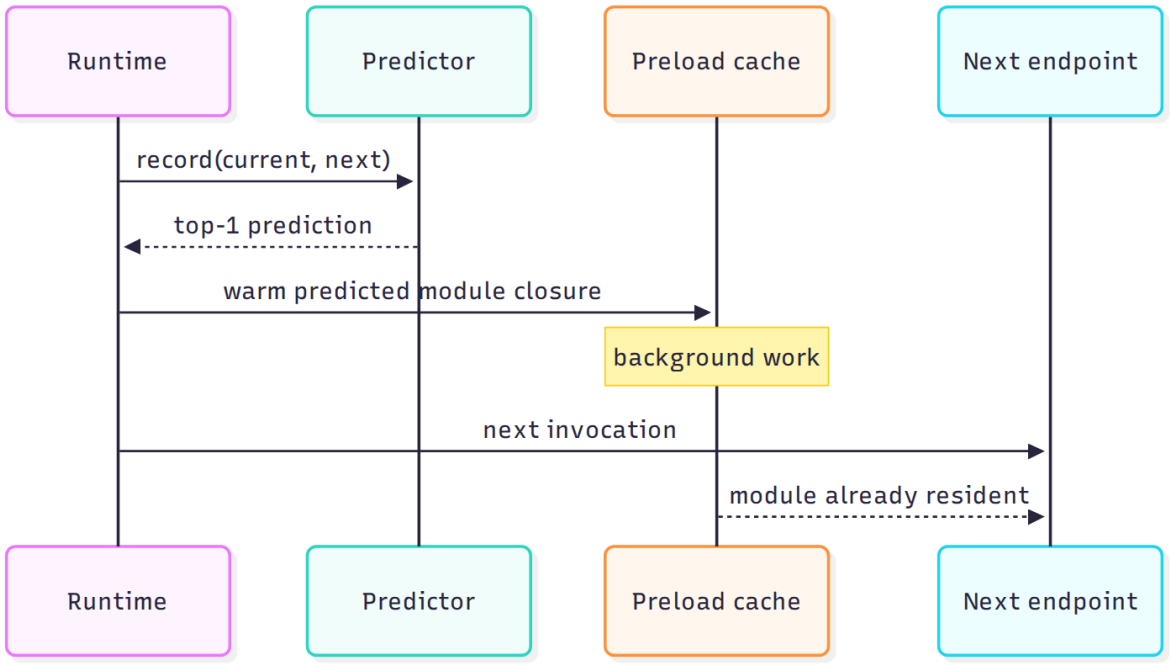


Fig. 2. Runtime loop used by PCSO: record a transition, predict the next endpoint, warm its dependency closure, and reuse the resident modules on the next invocation.

TABLE II
MEAN LATENCY COMPARISON AGAINST THE TRUE NO-OPTIMIZER BASELINE.

	Code loading (ms)			Total response (ms)		
Workload	**Baseline**	**V1**	**PCSO**	**Baseline**	**V1**	**PCSO**
Tier 1	3.62	2.05	0.40	5.08	3.50	1.88
Tier 2	44.32	24.76	5.37	49.61	30.02	10.63
Tier 3	81.15	17.05	5.46	89.47	88.98	43.52
Blended fleet	31.36	14.45	4.77	35.29	25.16	15.48

43.52 ms, a 51.4% reduction relative to the baseline and a 51.1% reduction relative to V1.

For the blended fleet, V1 reduces code-loading latency by 53.9% and total latency by 28.7%. PCSO improves those reductions to 84.8% and 56.1%. The tiered sequences produce higher prediction hit rates than the deliberately mixed fleet: 79.2% for Tier 1, 66.7% for Tier 2, 41.7% for Tier 3, and 12.5% for the blended profile.

V. DISCUSSION

The evaluation supports two conclusions. First, static dependency analysis is a useful and measurable baseline: it removes unnecessary startup work and gives the runtime an explicit dependency manifest. Second, static deferral

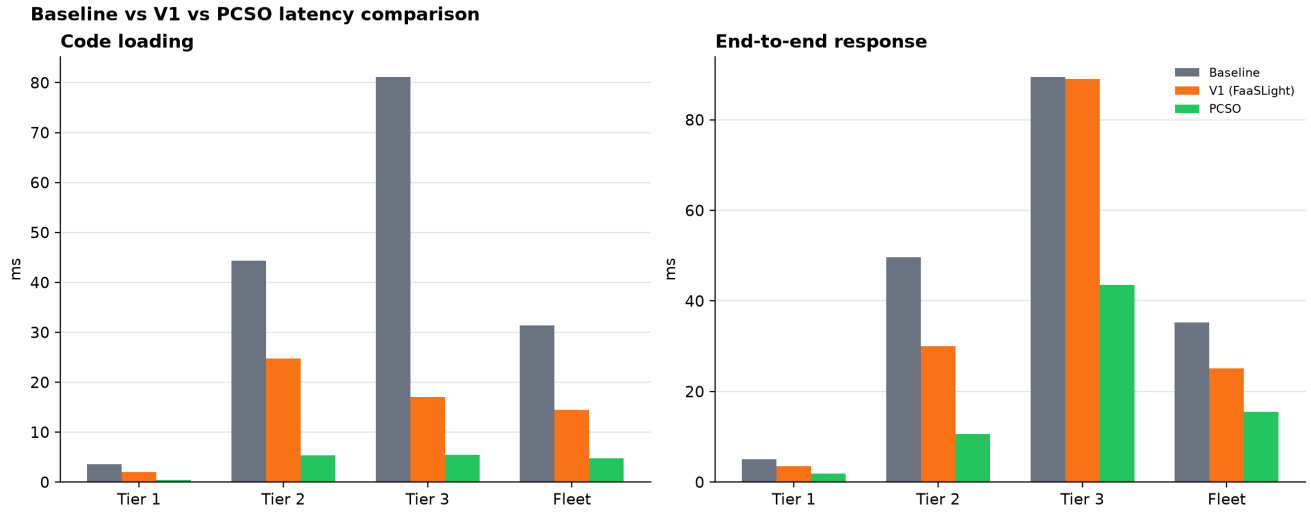


Fig. 3. Mean code-loading and end-to-end response latency for the eager baseline, V1/FaaSLight, and final PCSO.

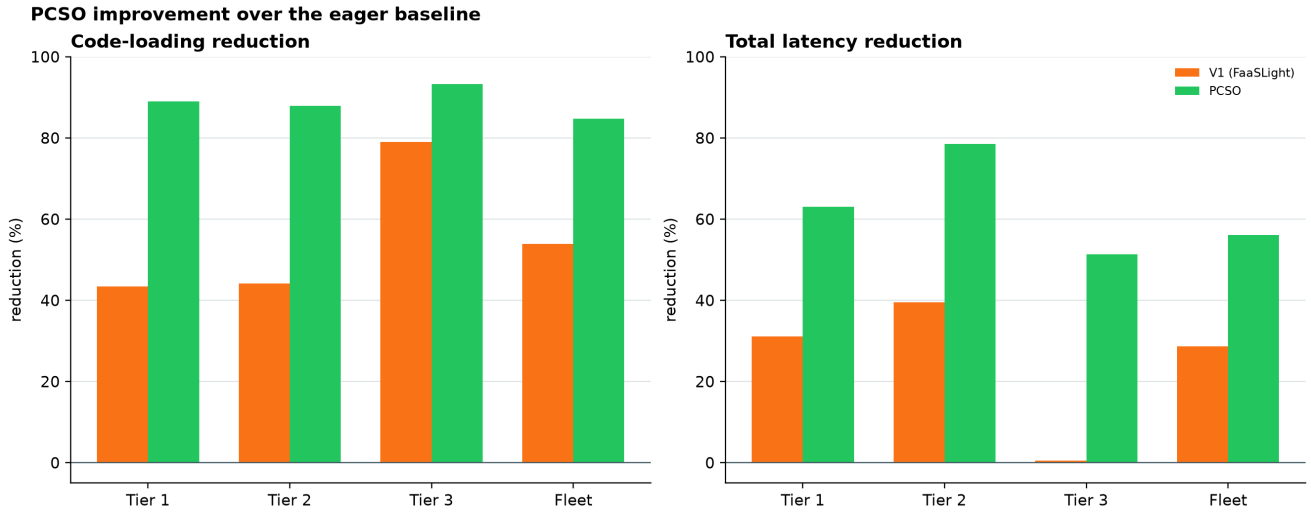


Fig. 4. Reduction relative to the true eager baseline. PCSO extends the V1 improvement by removing shifted handler-side import cost.

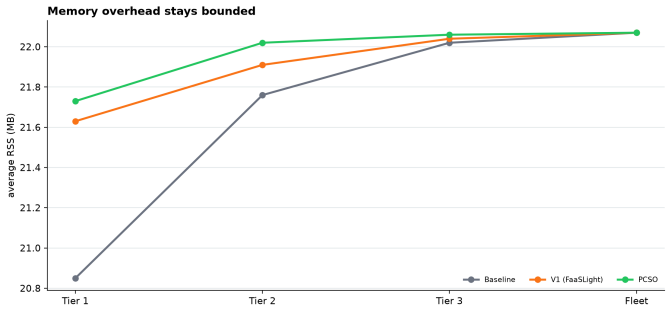


Fig. 5. Resident memory across the workload profiles for the three runtime versions.

alone does not guarantee a faster request. When a deferred dependency is almost certain to be used, moving its import into the handler only relocates the cost. PCSO addresses this case by using runtime sequence information to place the cost before the next request.

The improvement is workload-dependent. Predictable sequences give PCSO more opportunities to warm the correct closure, while a mixed fleet lowers prediction confidence. This is why the blended cache-hit rate is lower even though the blended total-latency improvement remains substantial.

VI. Limitations and Future Work

The evaluation is a controlled application-level benchmark rather than a measurement of a particular cloud provider's network, billing, or container scheduler. The dependency weights model relative initialization cost and should not be interpreted as provider-specific timings. The predictor is first-order and online; it can be inaccurate after an abrupt workload change. Future work should

evaluate production traces, add confidence decay, enforce a memory budget, and move preloading to an asynchronous worker with explicit resource limits.

VII. Conclusion

PCSO combines static dependency analysis, transition prediction, and dependency-closure preloading. Against the true eager baseline, it reduces blended code-loading latency by 84.8% and total response latency by 56.1%. The three-way comparison also clarifies the source of the gain: V1 removes optional imports from startup, while PCSO prevents heavy deferred imports from reappearing on the handler's critical path.

References

[1] Y. Liu *et al.*, "FaaSLight: General Application-Level Cold-Start Latency Optimization for Function-as-a-Service," arXiv:2207.08175, 2023.

[2] Amazon Web Services, "Understanding AWS Lambda scaling and cold starts," AWS documentation.